

Gitを使おう！！！！

UniPro Git 学習会

Akatsuki Yuito 2025/08/07

自己紹介

- 名前: あかつきゆいと
- UniPro創設者
- フルスタックエンジニア
- なんかセキュリティキャンプ行くってよ

TOC

Gitを使おう！

自己紹介

TOC

Gitとは？

Let's Try!

GitHub

リポジトリをGitHubに作ってみる

今日のまとめ

Gitとは？

Gitは一言で話すと、バージョン管理ツールです。

- ミスっても簡単に戻せる
- 他の人と作業するときに競合が起きにくい
- なぜこの仕様になったかというメモにも使える

Let's Try!

おそらく使いながら解説した方がわかりやすいので、とりあえず使ってみましょう。

設定

まずはGitの設定です。

ターミナルを開きましょう。

メールアドレスとユーザー名

メールアドレスとユーザー名をGitHubに登録したものにしましょう。(後でGitHubを使うときに、これ以外だと面倒なことになります。)

```
git config --global user.email hoge@exsample.com  
git config --global user.name hoge-user
```

リポジトリ

Gitでは、一つのプロジェクトをリポジトリというもので扱います。 リポジトリの実態はディレクトリです。

リポを初期化する(作ってみる)

リポジトリを作成するには、このコマンドを使います。

```
git init
```

コミット

コミットとは、変更を保存する処理やその保存された変更のことを指します。

Gitでは、このコミットをつなげていってバージョンを管理します。

コミットしてみる

コミットをするにはまず変更が必要ですね。 ファイルを作ってみましょう。

```
# hello.mdに"# Hello Git"を書き込む  
echo "# Hello Git" > hello.md
```

次に、**ステージング**します。

これは、どのファイルを一つのコミットに含めるかを指定します。

```
# hello.mdをステージングする  
git add hello.md
```

さて、いよいよコミットです。

コミットには、**コミットメッセージ**という、
こういったコミットなのかのメモ書きができます。

```
# Inital Commitというコミットを作成する  
git commit -m "Inital Commit"
```


ブランチ

ブランチとは、枝だと思ってください。これをどのように使っていくか、なぜ便利なのかは後で解説します。

デフォルトでは、mainもしくはmasterブランチというものが生えています。

ブランチを作ってみる

ブランチを作成するには、このコマンドを使います。

```
# testというブランチを作成している
git branch test
# testブランチへ移動する
git switch test
```

ブランチに変更を加えてみる

ブランチにある `hello.md` に変更を加えてみましょう。

VSCoDeで開いても何を用いても構いません。

変更をコミットする

先ほど同様にステージング/コミットします。

```
git add *  
git commit -m "test"
```

ブランチを切り替えてみる

ブランチを切り替えましょう。

切り替えてみると、それぞれのブランチごとに変更があり、分離されていることがわかります。

ブランチを切り替える

切り替えるには、`git switch` コマンドを使用します。

```
# mainブランチへ移動する  
git switch main
```

hello.md を確認してみよう

さっき編集したはずの `hello.md` を確認してみましょう。

変更されていないことがわかります。

ブランチを統合してみる

ブランチを統合します。これにより、`test` ブランチの変更が `main` ブランチと統合されて `main` ブランチに反映されます。

ブランチを統合する

統合するには、`git merge` コマンドを使用します。

```
# mainブランチへ切り替える
git switch main
# 今いるブランチ(main)へtestを統合する
git merge test
```

hello.md を確認してみよう

`hello.md` を確認してみましょう。

`test` ブランチの変更が適用されていることがわかります。

お疲れ様でした 🙌

これでGitの基本的な操作は終わりです！ これからは、これをどう使っていくかを書いていきます。

誰が編集したかがわかる

コミットにはあなたの情報が記録されているので、誰がコミットしたかがわかります。

```
git logs
```

```
commit 71a8d3d598b32a78ece9a4291722229858591119 (HEAD -> main, origin/main, origin/HEAD)  
Author: Yuito Akatsuki (Tani Yutaka) <yuito@yuito-it.jp>  
Date:   Mon Aug 4 21:04:32 2025 +0900
```

型定義忘れ

ミスっても簡単に戻せる

Gitはコミットを枝状に行なっていきます。

なので、特定の点(コミット)まで戻すことができます。

```
# コミットIDはgit logsやSourcetreeなどのGUIのツールから取得する  
git revert <コミットID>
```

GitHub

世の中にはGitリポを他人と共有するGitプロバイダーなるものがいくつかあります。

そのうちの 하나가GitHubです。

GitHubCLIのインストール

まずは、GitHubCLIをインストールします。

インストール手順は<https://github.com/cli/cli#installation>にあります。

Windows

```
winget install --id GitHub.cli
```

Mac

```
brew install gh
```

Linux

自分で調べてください()

GitHubへのログイン

次に、GitHubへログインします。

```
gh auth login
```

リポジトリをGitHubに作ってみる

Webから空のリポジトリを作る方法もありますが、今回は既存のローカルリポをアップロードしてみましょう。

```
gh repo create <リポ名> --private --source=. --remote=upstream
```

Webで試してみる

URLは

```
https://github.com/ユーザー名/リポジトリ名
```

です。

ローカルで変更してみる

ローカルで `hello.md` を編集してコミットしてみましょう。

適当に編集して、

```
git add *  
git commit -m "コミットメッセージ"
```

でしたね。

Pushしてみる

変更をアップロードしましょう。

アップロードのことを**Push**といいます。

```
git push --set-upstream upstream master
```

Webで変更を試みる

さっき開いたWeb版で変更してみましょう。

Pullしてみましょう

リモート(GitHub)の変更をローカルに落としてきましょう。

落とす作業を**Pull**といいます。

```
git Pull
```


Well done 🙌

お疲れ様でした。

今日のまとめ

- Gitはバージョン管理ツール
- コミットという形式でデータを保存していく
- 特定のコミットまで巻き戻すことができる
- 枝状にブランチを作成して、統合していくことができる
- GitHubではサーバーにリポを置いて他の人と共有することができる

またお会いしましょう 🖐️